

Lesson 02: Name Greeting

Lesson 01 printed the same greeting every time. In this lesson, a program will ask for your name, remember what you type, and use it in a new greeting.

Learning Goals

By the end of this lesson, you will be able to:

- identify a question printed by the program as **output**;
- enter text from the keyboard as **input**;
- describe a stored value as information kept in a labeled place; and
- explain the program's four focus actions: ask, receive, store, and greet.

Before You Begin

Complete [lesson 01: Hello, World](#). Use the same teacher-prepared Linux laptop, VS Code, terminal, and GCC compiler. You do not need an Arduino.

Core Mission: Give the Program Your Name

Goal: Run the supplied program, enter a name containing a space, and see that whole name in its greeting.

Prepare: Open VS Code. Use the supplied `main.c` without changing it.

1. Choose **File > Open Folder...** and select `coding-fundamentals-lessons/02-name-greeting`.
2. Choose **Terminal > New Terminal**.
3. Check the terminal's current folder:

```
pwd
```

The path must end with `coding-fundamentals-lessons/02-name-greeting`.

4. List the lesson files:

```
ls
```

Confirm that you see `main.c`. You might also see `name-greeting` if the program was built before.

5. Build the supplied program:

```
gcc -std=c11 -Wall -Wextra -Werror main.c -o name-greeting
```

A successful build usually returns to the prompt without printing a message.

6. Run the program:

```
./name-greeting
```

7. When the program asks `What is your name?`, type a name with at least two words, such as your first and last name. Press Enter.

Notice: Record the interaction you observed.

Question printed by the program: _____

Name entered from the keyboard: _____

Greeting printed by the program: _____

Did the complete name, including the space, appear in the greeting? **Yes / No**

Stop when the terminal prompt returns. You have completed the Core Mission. Leave `main.c` unchanged.

Name the Four Focus Actions

The program followed four important actions:

ASK		RECEIVE		STORE	
GREET					
print a question	—>	get keyboard input	—>	keep it in name	—> print the greeting

1. **Ask:** Print a question for the person using the program.
2. **Receive:** Get the text typed on the keyboard. That text is **input**.
3. **Store:** Keep the input in a labeled place named `name`.
4. **Greet:** Print fixed words plus the value stored in `name`. Printed text is **output**.

Think of `name` as a labeled box inside the running program:

```
label: name
value: Ada Lovelace
```

The value can be different each time the program runs, but the label stays `name`.

Connect the Actions to the Code

Open `main.c` and find these focus lines:

Focus action	Code to find	What to notice
Ask	<code>printf("What is your name? ");</code>	Prints the question.
Receive	<code>fgets(name, sizeof(name), stdin)</code>	Receives keyboard input and stores it in <code>name</code> . The extra parts help limit the input safely.
Store	<code>char name[50];</code> and <code>name</code> inside <code>fgets(...)</code>	The first line prepares the labeled place. <code>fgets</code> puts the received value there. The size details are safety helpers.
Greet	<code>printf("Nice to meet you, %s!\n", name);</code>	Puts the stored name where <code>%s</code> appears in the greeting.

For this lesson, explain what the four actions do. You do not need to memorize their punctuation or explain every part of `fgets`.

Complete Safe Program

This is the full supplied source:

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char name[50];

    printf("What is your name? ");
    fflush(stdout);

    if (fgets(name, sizeof(name), stdin) == NULL) {
        printf("\nNo name was entered.\n");
        return 1;
    }

    name[strcspn(name, "\n")] = '\0';
    if (name[0] == '\0') {
        printf("Please run the program again and type at least one character.\n");
        return 1;
    }

    printf("Nice to meet you, %s!\n", name);
    return 0;
}
```

The program contains more code than the four ideas you are learning. That extra code lets the program handle input safely and honestly.

Helper Mechanics

These helpers are part of the real program, but they are not lesson questions to memorize:

Helper	What it provides
Bounded input	<code>name[50]</code> and <code>sizeof(name)</code> limit how much text goes into the storage space.
Visible question	<code>fflush(stdout)</code> makes sure the question appears before the program waits for input.
End-of-input handling	The first <code>if</code> notices when no input can be received and stops with an honest message.
Newline cleanup	The <code>strcspn</code> line removes the Enter-key newline from the stored name before printing the greeting.
Empty-name handling	The second <code>if</code> asks the user to try again instead of printing a greeting with no name.
Program finish	The <code>return</code> lines report whether the program completed its normal greeting or stopped early.

You may inspect these helpers or ask how they work. To complete this lesson, you only need to explain **ask, receive, store, and greet**.

Try One Different Input

You have seen the program work, so now make a prediction from that evidence.

1. Imagine running the unchanged program and entering `Ada Lovelace`.
2. **Predict:** Write the exact final greeting.

Prediction: _____

3. Run the program again:

```
./name-greeting
```

4. Enter `Ada Lovelace` and press Enter.
5. Record the final greeting:
Actual output: _____
6. **Explain:** Which words were fixed by the program, and which words came from the stored input?

Fixed words: _____

Stored input: _____

My prediction was the same as / different from the result because

Optional Cleanup

Remove only the generated executable with:

```
rm name-greeting
```

The source file `main.c` remains and can build the executable again.

Troubleshooting

- **`main.c` does not appear after `ls`:** Reopen the `02-name-greeting` folder before building.
- **`gcc` is not found:** Ask the teacher to prepare the compiler.
- **`GCC` reports an error:** Read the first error with the teacher and compare `main.c` with the supplied source.
- **`./name-greeting` is missing:** Check that the build returned without an error and that `ls` shows `name-greeting`.
- **Only part of a very long name appears:** The storage has a safe size limit. Use a shorter name for this activity.
- **The program asks for at least one character:** Run it again and type a name before pressing Enter.
- **The program prints `No name was entered.`:** Run it again in the terminal and enter a name when it asks.

Next, continue to [lesson 03: Repeating Counter](#).